LAB ASSIGNMENT 13.2

Gundekari Yeshashwini
2303A52023
Batch - 38

Task Description -1 (Refactoring – Eliminating Repetitive Logic)
• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.
Instructions:
• Ask AI to locate repeated computations within the program.
• Encapsulate repeated logic into reusable functions.
• Verify that program output remains unchanged after refactoring.
• Add proper docstrings to all defined functions.
Sample Code:
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
Expected Output:
A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Prompt:

```
# Refactor this Python code to remove repeated calculations.
# Create a reusable function to compute the area and perimeter of a rectangle.
# Ensure the output remains the same.
# Add proper docstrings to the function.

# length = 8
# width = 4
# print("Area:", length * width)
# print("Perimeter:", 2 * (length + width))

# length = 12
# width = 6
# print("Area:", length * width)
```

Code :

```python
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter
# First rectangle
length1 = 8
width1 = 4
area1, perimeter1 = calculate_rectangle_properties(length1, width1)
print("Area:", area1)
print("Perimeter:", perimeter1)
# Second rectangle
length2 = 12
width2 = 6
area2, perimeter2 = calculate_rectangle_properties(length2, width2)
print("Area:", area2)
print("Perimeter:", perimeter2)
```

Output:

```
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Task Description -2 (Refactoring – Modularizing Inline
Calculations)
• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.
Instructions:
• Identify repeated calculation patterns in the code.
• Convert them into reusable functions.
• Ensure readability and proper documentation.
Sample Code:
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
Expected Output:
A modular Python program that uses a reusable function to compute final amounts for different inputs.

Prompt:

```
# Refactor this Python code to remove repeated inline calculations.
# Create a reusable function that calculates the final amount after a 10% discount.
# Ensure the output remains the same.
# Add a proper docstring explaining the function.

# amount = 1000
# discount = amount * 0.10
# final_amount = amount - discount
# print("Final Amount:", final_amount)

# amount = 2000
# discount = amount * 0.10
# final_amount = amount - discount
# print("Final Amount:", final_amount)
```

Code:

```python
def calculate_final_amount(amount):
    """
    Calculate the final amount after applying a 10% discount.

    Parameters:
    amount (float): The original amount before discount.

    Returns:
    float: The final amount after applying the discount.
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount
# Example usage
amount1 = 1000
final_amount1 = calculate_final_amount(amount1)
print("Final Amount:", final_amount1)
amount2 = 2000
final_amount2 = calculate_final_amount(amount2)
print("Final Amount:", final_amount2)
```

Output:

```
xtensions\ms-python.debugpy-2025.18.0-win32-x64
Final Amount: 900.0
Final Amount: 1800.0
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
```

else:
print(name, "not found")

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Prompt:

```
# Optimize the following Python code by removing nested loops and using a faster lookup method such as a set.
# Preserve the program output and add a small execution time comparison between the original and optimized versions.
# Explain the improvement briefly in comments.

# students = ["Anil", "Bhavya", "Charan", "Divya"]
# check_list = ["Charan", "Esha", "Bhavya"]
# for name in check_list:
# exists = False
# for student in students:
# if name == student:
# exists = True
# if exists:
# print(name, "found")
# else:
# print(name, "not found")
```

Code:

```python
import time
# Original code with nested loops
def original_code(students, check_list):
    for name in check_list:
        exists = False
        for student in students:
            if name == student:
                exists = True
                break  # Break to avoid unnecessary iterations
        if exists:
            print(name, "found")
        else:
            print(name, "not found")
# Optimized code using a set for faster lookup
def optimized_code(students, check_list):
    student_set = set(students)  # Convert list to set for O(1) average time complexity
    for name in check_list:
        if name in student_set:  # Check membership in the set
            print(name, "found")
        else:
            print(name, "not found")
```

```python
# Sample data
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
# Measure execution time for original code
start_time = time.time()
original_code(students, check_list)
end_time = time.time()
print("Original code execution time:", end_time - start_time, "seconds")
# Measure execution time for optimized code
start_time = time.time()
optimized_code(students, check_list)
end_time = time.time()
print("Optimized code execution time:", end_time - start_time, "seconds")
# Improvement Explanation:
# The original code uses nested loops to check for the presence of each name in the students list
# which results in O(n*m) time complexity (where n is the length of check_list and m is the length of students).
# The optimized code converts the students list into a set, allowing for O(1) average time complexity for membership checks,
# thus reducing the overall time complexity to O(n) for checking all names in the check_list
```

Output:

```
Charan found
Esha not found
Bhavya found
Original code execution time: 0.0007033348083496094 seconds
Charan found
Esha not found
Bhavya found
Optimized code execution time: 0.00028896331787109375 seconds
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

Prompt:

```
# Refactor this Python code by replacing hardcoded values with named constants.
# Define constants like RATE and TIME at the top of the program.
# Update the calculations to use these constants while keeping the output unchanged.

# print("Simple Interest:", (1000 * 2 * 5) / 100)
# print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Code:

```
# Define constants for rate and time
RATE = 5  # Interest rate in percentage
TIME = 2  # Time in years
# Calculate simple interest using constants
def calculate_simple_interest(principal):
    return (principal * TIME * RATE) / 100
# Sample principal amounts
principal1 = 1000
principal2 = 2000
# Calculate and print simple interest for both principal amounts
print("Simple Interest for", principal1, ":", calculate_simple_interest(principal1))
print("Simple Interest for", principal2, ":", calculate_simple_interest(principal2))
# Improvement Explanation:
# By defining RATE and TIME as constants at the top of the program, we improve code readability and maintainability.
# If we need to change the interest rate or time period in the future, we can simply
# update the constants without having to search through the code for hardcoded values, reducing the risk of errors.
```

Output:

```
Simple Interest for 1000 : 100.0
Simple Interest for 2000 : 200.0
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Prompt:

```python
# Refactor this Python code by improving variable naming and code clarity.
# Replace unclear variable names with meaningful identifiers and add inline comments.
# Ensure the program output remains unchanged.

# x = 50
# y = 30
# z = x + y
# print(z)
```

Code:

```python
# Define meaningful variable names for the numbers being added
number1 = 50  # First number to be added
number2 = 30  # Second number to be added
# Calculate the sum of the two numbers
result = number1 + number2
# Print the result of the addition
print(result)
# Improvement Explanation:
# By replacing the variable names x, y, and z with number1, number2,
# and result, we enhance the readability of the code. The new variable names clearly indicate their purpose,
# making it easier for others to understand the code at a glance. Additionally, adding inline comments provides
# context for each step of the calculation, further improving code clarity.
```

Output:

```
80
```